VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

On

OPERATING SYSTEMS (23CS4PCOPS)

Submitted by

ANSHIKA GUPTA

(1BF24CS058)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
February to June 2026

CERTIFICATE

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

This is to certify that the Lab work entitled **"OPERATING SYSTEMS"** carried out by **Anshika Gupta (1BF24CS058)**, who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Operating Systems Lab **(23CS4PCOPS)** work prescribed for the said degree.

Rajeshwari K
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Course outcomes:

CO1	Apply the different concepts and functionalities of Operating System.
CO2	Analyse various Operating system strategies and techniques.
CO3	Demonstrate the different functionalities of Operating System
CO4	Conduct practical experiments to implement the functionalities of Operating system.

Lab program 1:

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

a) FCFS

CODE:

```
#include <stdio.h>

int main() {
   int n, bt[20], wt[20], tat[20], at[20], ct[20], pid[20];
   int i, j, temp;
   float twt = 0.0, ttat = 0.0, awt, att;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++) {
      pid[i] = i + 1;
      printf("Enter Arrival Time for P%d: ", i + 1);
      scanf("%d", &at[i]);
      printf("Enter Burst Time for P%d: ", i + 1);
      scanf("%d", &bt[i]);
   }
   for(i = 0; i < n - 1; i++) {
      for(j = i + 1; j < n; j++) {
         if(at[i] > at[j]) {
            // Swap arrival times
            temp = at[i]; at[i] = at[j]; at[j] = temp;
            // Swap burst times
            temp = bt[i]; bt[i] = bt[j]; bt[j] = temp;
            // Swap process IDs
            temp = pid[i]; pid[i] = pid[j]; pid[j] = temp;
         }
      }
   }

 ct[0] = at[0] + bt[0];
   tat[0] = ct[0] - at[0];
   wt[0] = tat[0] - bt[0];
```

```
    for(i = 1; i < n; i++) {
        if(ct[i - 1] < at[i]) {
            ct[i] = at[i] + bt[i];
        } else {
            ct[i] = ct[i - 1] + bt[i];
        }
        tat[i] = ct[i] - at[i];
        wt[i] = tat[i] - bt[i];
    }
    for(i = 0; i < n; i++) {
        twt += wt[i];
        ttat += tat[i];
    }
    awt = twt / n;
    att = ttat / n;
    printf("\nP\tAT\tBT\tCT\tWT\tTAT");
    for(i = 0; i < n; i++) {
        printf("\nP%d\t%d\t%d\t%d\t%d\t%d",
              pid[i], at[i], bt[i], ct[i], wt[i], tat[i]);
    }
    printf("\n\nAverage Waiting Time: %.2f", awt);
    printf("\nAverage Turnaround Time: %.2f\n", att);

    return 0;
}
```

Output

```
Enter number of processes: 3
Enter Arrival Time for P1: 0
Enter Burst Time for P1: 24
Enter Arrival Time for P2: 0
Enter Burst Time for P2: 3
Enter Arrival Time for P3: 0
Enter Burst Time for P3: 3

P       AT      BT      CT      WT      TAT
P1      0       24      24      0       24
P2      0       3       27      24      27
P3      0       3       30      27      30

Average Waiting Time: 17.00
Average Turnaround Time: 27.00

Process returned 0 (0x0)   execution time : 15.553 s
Press any key to continue.
```

b) SJF (Preemptive)

CODE

```
#include <stdio.h>

int main() {
   int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10], rt[10];
   int i, time = 0, completed = 0, smallest, prev = -1;
   float avg_wt = 0, avg_tat = 0;


   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++) {
      p[i] = i + 1;
      printf("\nProcess %d\n", i+1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
      bt_copy[i] = bt[i];
      rt[i] = bt[i];
   }

   printf("\nGantt Chart: ");

   while(completed < n) {
      smallest = -1;
      int min_rt = 9999;
      for(i = 0; i < n; i++) {
         if(at[i] <= time && rt[i] > 0 && rt[i] < min_rt) {
            min_rt = rt[i];
            smallest = i;
         }
      }

      if(smallest == -1) {
         time++;
         continue;
      }
```

```
        if(prev != p[smallest]) {
            printf("P%d ", p[smallest]);
            prev = p[smallest];
        }
        rt[smallest]--;
        time++;
        if(rt[smallest] == 0) {
            ct[smallest] = time;
            tat[smallest] = ct[smallest] - at[smallest];
            wt[smallest] = tat[smallest] - bt[smallest];
            completed++;
        }
    }

    printf("\n\nP\tAT\tBT\tCT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i], at[i], bt_copy[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3

Process 1
Arrival Time: 10
Burst Time: 2

Process 2
Arrival Time: 10
Burst Time: 2

Process 3
Arrival Time: 3
Burst Time: 9

Gantt Chart: P3 P1 P2 P3

P       AT      BT      CT      WT      TAT
P1      10      2       12      0       2
P2      10      2       14      2       4
P3      3       9       16      4       13

Average Waiting Time: 2.00
Average Turnaround Time: 6.33

Process returned 0 (0x0)   execution time : 30.911 s
Press any key to continue.
```

b) SJF (Non-Preemptive)

CODE

```
#include <stdio.h>

int main() {
    int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10];
    int i, j, time = 0, completed = 0, smallest;
    float avg_wt = 0, avg_tat = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("Process %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        bt_copy[i] = bt[i];
    }
    while(completed < n) {
        smallest = -1;
        for(i = 0; i < n; i++) {
```

```
            if(at[i] <= time && bt[i] > 0) {
                if(smallest == -1 || bt[i] < bt[smallest]) {
                    smallest = i;
                }
            }
        }
        if(smallest == -1) {
            time++;
            continue;
        }
        time += bt[smallest];
        ct[smallest] = time;
        tat[smallest] = ct[smallest] - at[smallest];
        wt[smallest] = tat[smallest] - bt[smallest];

        bt[smallest] = 0;
        completed++;
    }

    printf("\nP\tAT\tBT\tCT\tWT\tTAT\n");

    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
                p[i], at[i], bt_copy[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3
Process 1
Arrival Time: 10
Burst Time: 2
Process 2
Arrival Time: 10
Burst Time: 2
Process 3
Arrival Time: 3
Burst Time: 9

P       AT      BT      CT      WT      TAT
P1      10      2       14      2       4
P2      10      2       16      4       6
P3      3       9       12      0       9

Average Waiting Time: 2.00
Average Turnaround Time: 6.33

Process returned 0 (0x0)   execution time : 8.111 s
Press any key to continue.
```

c) Priority (Preemptive)

CODE

```
#include <stdio.h>

int main() {
    int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10], pr[10], rt[10];
    int i, time = 0, completed = 0, highest, prev = -1;
    float avg_wt = 0, avg_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for(i = 0; i < n; i++) {
        p[i] = i + 1;
        printf("\nProcess %d\n", i+1);
        printf("Arrival Time: ");
        scanf("%d", &at[i]);
        printf("Burst Time: ");
        scanf("%d", &bt[i]);
        printf("Priority (lower number = higher priority): ");
        scanf("%d", &pr[i]);
```

```
    bt_copy[i] = bt[i];
    rt[i] = bt[i];
  }

  printf("\nGantt Chart: ");

  while(completed < n) {
    highest = -1;
    int max_pr = 9999;
    for(i = 0; i < n; i++) {
      if(at[i] <= time && rt[i] > 0 && pr[i] < max_pr) {
        max_pr = pr[i];
        highest = i;
      }
    }

    if(highest == -1) {
      time++;
      continue;
    }

    if(prev != p[highest]) {
      printf("P%d ", p[highest]);
      prev = p[highest];
    }
    rt[highest]--;
    time++;
    if(rt[highest] == 0) {
      ct[highest] = time;
      tat[highest] = ct[highest] - at[highest];
      wt[highest] = tat[highest] - bt[highest];
      completed++;
    }
  }

  printf("\n\nP\tAT\tBT\tPR\tCT\tWT\tTAT\n");
  for(i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
          p[i], at[i], bt_copy[i], pr[i], ct[i], wt[i], tat[i]);
    avg_wt += wt[i];
    avg_tat += tat[i];
  }
```

```
    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3

Process 1
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 3

Process 2
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 1

Process 3
Arrival Time: 3
Burst Time: 9
Priority (lower number = higher priority): 2

Gantt Chart: P3 P2 P3 P1

P       AT      BT      PR      CT      WT      TAT
P1      10      2       3       16      4       6
P2      10      2       1       12      0       2
P3      3       9       2       14      2       11

Average Waiting Time: 2.00
Average Turnaround Time: 6.33

Process returned 0 (0x0)   execution time : 142.278 s
Press any key to continue.
```

c) Priority (Non-Preemptive)

CODE

```
#include <stdio.h>

int main() {
   int n, bt[10], bt_copy[10], wt[10], tat[10], at[10], ct[10], p[10], pr[10];
   int i, j, time = 0, completed = 0, highest;
   float avg_wt = 0, avg_tat = 0;

   printf("Enter number of processes: ");
   scanf("%d", &n);

   for(i = 0; i < n; i++) {
      p[i] = i + 1;
      printf("\nProcess %d\n", i+1);
      printf("Arrival Time: ");
      scanf("%d", &at[i]);
      printf("Burst Time: ");
      scanf("%d", &bt[i]);
      printf("Priority (lower number = higher priority): ");
      scanf("%d", &pr[i]);
      bt_copy[i] = bt[i];
   }

   printf("\nGantt Chart: ");

   while(completed < n) {
      highest = -1;
      int max_pr = 9999;
      for(i = 0; i < n; i++) {
         if(at[i] <= time && bt[i] > 0 && pr[i] < max_pr) {
            max_pr = pr[i];
            highest = i;
         }
      }

      if(highest == -1) {
         time++;
         continue;
      }
```

```
printf("P%d ", p[highest]);
        time += bt[highest];
        ct[highest] = time;
        tat[highest] = ct[highest] - at[highest];
        wt[highest] = tat[highest] - bt[highest];

        bt[highest] = 0;
        completed++;
    }

    printf("\n\nP\tAT\tBT\tPR\tCT\tWT\tTAT\n");
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
              p[i], at[i], bt_copy[i], pr[i], ct[i], wt[i], tat[i]);
        avg_wt += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f", avg_wt/n);
    printf("\nAverage Turnaround Time: %.2f\n", avg_tat/n);

    return 0;
}
```

Output

```
Enter number of processes: 3

Process 1
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 3

Process 2
Arrival Time: 10
Burst Time: 2
Priority (lower number = higher priority): 1

Process 3
Arrival Time: 3
Burst Time: 9
Priority (lower number = higher priority): 2

Gantt Chart: P3 P2 P1

P       AT      BT      PR      CT      WT      TAT
P1      10      2       3       16      4       6
P2      10      2       1       14      2       4
P3      3       9       2       12      0       9

Average Waiting Time: 2.00
Average Turnaround Time: 6.33
```

d) Round Robin

CODE

```
#include <stdio.h>

int main() {
    int n, quantum, i, time = 0, remain, flag = 0;
    int bt[10], at[10], wt[10], tat[10], rt[10], ct[10];
    int total_wt = 0, total_tat = 0;
    int gantt[100], gantt_time[100], gc_index = 0;

    printf("Enter total number of processes: ");
    scanf("%d", &n);
    remain = n;

    for(i = 0; i < n; i++) {
        printf("Enter Arrival Time and Burst Time for process P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter Time Quantum: ");
    scanf("%d", &quantum);

    printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");

    for(i = 0; remain != 0;) {
        if(at[i] <= time && rt[i] > 0) {
            if(rt[i] <= quantum) {
                time += rt[i];
                rt[i] = 0;
                remain--;
                ct[i] = time;
                tat[i] = ct[i] - at[i];
                wt[i] = tat[i] - bt[i];
                total_wt += wt[i];
                total_tat += tat[i];
                printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], tat[i],
wt[i]);
```

```
gantt[gc_index] = i+1;
                gantt_time[gc_index] = time;
                gc_index++;
                flag = 1;
            } else {
                rt[i] -= quantum;
                time += quantum;
                gantt[gc_index] = i+1;
                gantt_time[gc_index] = time;
                gc_index++;
                flag = 1;
            }
        }

        i++;
        if(i == n) {
            if(flag == 0) {
                time++;
            }
            i = 0;
            flag = 0;
        }
    }
    printf("\nGantt Chart:\n|");
    for(i = 0; i < gc_index; i++) {
        printf(" P%d |", gantt[i]);
    }
    printf("\n0");
    for(i = 0; i < gc_index; i++) {
        printf("   %d", gantt_time[i]);}
    printf("\n\nAverage Turnaround Time = %.2f", (float)total_tat/n);
    printf("\nAverage Waiting Time = %.2f\n", (float)total_wt/n);
    return 0;
}
```

Output

```
Enter total number of processes: 6
Enter Arrival Time and Burst Time for process P1: 5 5
Enter Arrival Time and Burst Time for process P2: 4 6
Enter Arrival Time and Burst Time for process P3: 3 7
Enter Arrival Time and Burst Time for process P4: 1 9
Enter Arrival Time and Burst Time for process P5: 2 2
Enter Arrival Time and Burst Time for process P6: 3 6
Enter Time Quantum: 4

Process AT      BT      CT      TAT     WT
P5      2       2       7       5       3
P6      3       6       29      26      20
P1      5       5       30      25      20
P2      4       6       32      28      22
P3      3       7       35      32      25
P4      1       9       36      35      26

Gantt Chart:
| P4 | P5 | P6 | P1 | P2 | P3 | P4 | P6 | P1 | P2 | P3 | P4 |
0   5   7   11   15   19   23   27   29   30   32   35   36

Average Turnaround Time = 25.17
Average Waiting Time = 19.33

Process returned 0 (0x0)   execution time : 18.206 s
Press any key to continue.
```

```
Enter total number of processes: 3
Enter Arrival Time and Burst Time for process P1: 2 1
Enter Arrival Time and Burst Time for process P2: 4 3
Enter Arrival Time and Burst Time for process P3: 6 5
Enter Time Quantum: 3

Process AT      BT      CT      TAT     WT
P1      2       1       3       1       0
P2      4       3       7       3       0
P3      6       5       12      6       1

Gantt Chart:
| P1 | P2 | P3 | P3 |
0   3   7   10   12

Average Turnaround Time = 3.33
Average Waiting Time = 0.33

Process returned 0 (0x0)   execution time : 12.655 s
Press any key to continue.
```

Program 2:

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories –system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

CODE:

```
#include <stdio.h>

int main() {
   int i, n_sys, n_user;


   int bt_sys[10], wt_sys[10], tat_sys[10];

   int bt_user[10], wt_user[10], tat_user[10];


   printf("Enter number of SYSTEM processes: ");
   scanf("%d", &n_sys);

   printf("Enter burst times for SYSTEM processes:\n");
   for(i = 0; i < n_sys; i++) {
      printf("P%d: ", i+1);
      scanf("%d", &bt_sys[i]);
   }


   printf("\nEnter number of USER processes: ");
   scanf("%d", &n_user);

   printf("Enter burst times for USER processes:\n");
   for(i = 0; i < n_user; i++) {
      printf("P%d: ", i+1);
      scanf("%d", &bt_user[i]);
   }
```

```
    wt_sys[0] = 0;
    for(i = 1; i < n_sys; i++) {
        wt_sys[i] = wt_sys[i-1] + bt_sys[i-1];
    }

    for(i = 0; i < n_sys; i++) {
        tat_sys[i] = wt_sys[i] + bt_sys[i];
    }


    int total_sys_time = 0;
    for(i = 0; i < n_sys; i++) {
        total_sys_time += bt_sys[i];
    }

    wt_user[0] = total_sys_time;
    for(i = 1; i < n_user; i++) {
        wt_user[i] = wt_user[i-1] + bt_user[i-1];
    }

    for(i = 0; i < n_user; i++) {
        tat_user[i] = wt_user[i] + bt_user[i];
    }


    printf("\nSYSTEM PROCESSES:\n");
    printf("Process\tBT\tWT\tTAT\n");
    for(i = 0; i < n_sys; i++) {

        printf("P%d\t%d\t%d\t%d\n", i+1, bt_sys[i], wt_sys[i], tat_sys[i]);
    }

    printf("\nUSER PROCESSES:\n");
    printf("Process\tBT\tWT\tTAT\n");
    for(i = 0; i < n_user; i++)
{
        printf("P%d\t%d\t%d\t%d\n", i+1, bt_user[i], wt_user[i], tat_user[i]);
    }

    return 0;
}
```

Output

```
Enter number of SYSTEM processes: 2
Enter burst times for SYSTEM processes:
P1: 3
P2: 5

Enter number of USER processes: 3
Enter burst times for USER processes:
P1: 4
P2: 2
P3: 6

SYSTEM PROCESSES:
Process BT      WT      TAT
P2      5       3       8

USER PROCESSES:
Process BT      WT      TAT
P2      2       12      14
P3      6       14      20
```

Program 3:

Write a C program to simulate Real-Time CPU Scheduling algorithms:

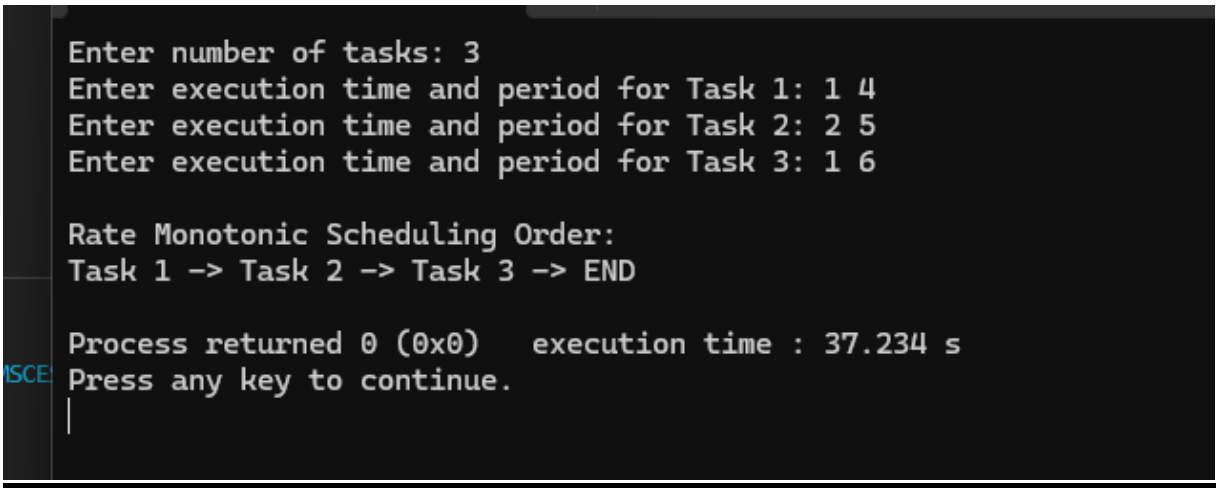
a) Rate-Monotonic

CODE:

```
#include <stdio.h>
struct Task {
int id, execution, period;
};

int main() {
int n, i, j, time = 0;
struct Task t[10], temp;

printf("Enter number of tasks: ");
scanf("%d", &n);

for(i = 0; i < n; i++) {
t[i].id = i + 1;
printf("Enter execution time and period for Task %d: ", i+1);
scanf("%d %d", &t[i].execution, &t[i].period);
}

// Sort by shortest period
for(i = 0; i < n-1; i++) {
for(j = i+1; j < n; j++) {
if(t[i].period > t[j].period) {
temp = t[i];
t[i] = t[j];
t[j] = temp;
}
}
}

printf("\nGantt Chart:\n|");
for(i = 0; i < n; i++) {
printf(" T%d |", t[i].id);
```

```
}

printf("\n0");
for(i = 0; i < n; i++) {
time += t[i].execution;
printf(" %d", time);
}

return 0;
}
```

Output



b) Earliest-deadline First

CODE:

```
#include <stdio.h>
struct Task {
int id, execution, deadline;
};
int main() {
int n, i, j, time = 0;
struct Task t[10], temp;

printf("Enter number of tasks: ");
scanf("%d", &n);
for(i = 0; i < n; i++) {
t[i].id = i + 1;
```

```
printf("Enter execution time and deadline for Task %d: ", i+1);
scanf("%d %d", &t[i].execution, &t[i].deadline);
}

// Sort by earliest deadline
for(i = 0; i < n-1; i++) {
for(j = i+1; j < n; j++) {
if(t[i].deadline > t[j].deadline) {
temp = t[i];
t[i] = t[j];
t[j] = temp;
}
}
}

printf("\nGantt Chart:\n|");
for(i = 0; i < n; i++) {
printf(" T%d |", t[i].id);
}

printf("\n0");
for(i = 0; i < n; i++) {
time += t[i].execution;
printf(" %d", time);
}

return 0;
}
```

Output

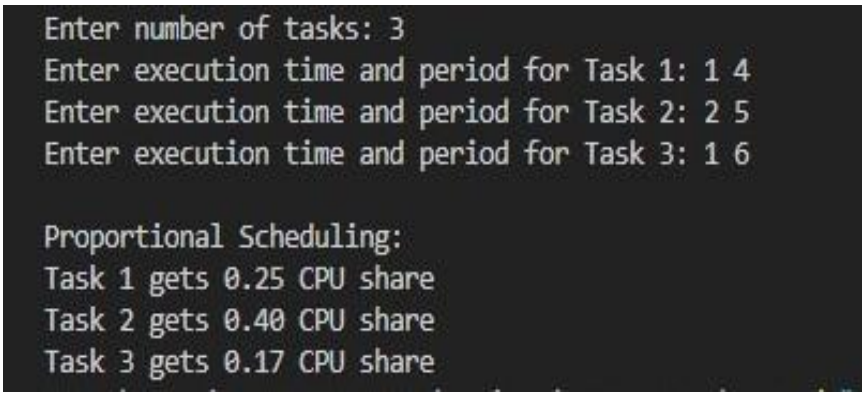
c) Proportional scheduling

CODE:

```
#include <stdio.h>

struct Task {
int id, execution, period;
float share;
};

int main() {
int n, i, totalTime = 10, allocated;
struct Task t[10];


printf("Enter number of tasks: ");
scanf("%d", &n);


for(i = 0; i < n; i++) {
t[i].id = i + 1;

printf("Enter execution time and period for Task %d: ", i+1);
scanf("%d %d", &t[i].execution, &t[i].period);

t[i].share = (float)t[i].execution / t[i].period;
}

printf("\nGantt Chart:\n|");
int current = 0;
for(i = 0; i < n; i++)
{
allocated = (int)(t[i].share * totalTime);
printf(" T%d |", t[i].id);
}

printf("\n0");
for(i = 0; i < n; i++)

{
```

```
allocated = (int)(t[i].share * totalTime);
current += allocated;
printf(" %d", current);

}

return 0;
}
```

Output



Program 4:

Write a C program to simulate:
a) Producer-Consumer problem using semaphores.

CODE

```
#include <stdio.h>
#include <stdlib.h>

int mutex = 1, full = 0, empty = 3, x = 0;

// wait operation
void wait() {
--mutex;
}

// signal operation
void signal() {
++mutex;
}

// producer
void producer() {
wait();
++full;
--empty;
x++;
printf("Producer has produced: Item %d\n", x);
signal();
}

// consumer
void consumer() {
wait();
--full;
++empty;
printf("Consumer has consumed: Item %d\n", x);
x--;
signal();
}
```

```
int main() {
int choice;

while (1) {
printf("\n1. Produce\n2. Consume\n3. Exit\n");
printf("Enter your choice: ");
scanf("%d", &choice);

switch (choice) {
case 1:
if ((mutex == 1) && (empty != 0))
producer();
else
printf("Buffer is full!\n");
break;

case 2:
if ((mutex == 1) && (full != 0))
consumer();
else
printf("Buffer is empty!\n");
break;

case 3:

exit(0);
}

}
```

Output

```
1. Produce
2. Consume
3. Exit
Enter your choice: 1
Producer has produced: Item 1

1. Produce
2. Consume
3. Exit
Enter your choice: 1
Producer has produced: Item 2

1. Produce
2. Consume
3. Exit
Enter your choice: 1
Producer has produced: Item 3

1. Produce
2. Consume
3. Exit
Enter your choice: 2
Consumer has consumed: Item 3

1. Produce
2. Consume
3. Exit
Enter your choice: 2
Consumer has consumed: Item 2

1. Produce
2. Consume
3. Exit
Enter your choice: 2
Consumer has consumed: Item 1

1. Produce
2. Consume
3. Exit
Enter your choice: 2
Buffer is empty!

1. Produce
2. Consume
3. Exit
Enter your choice:
```

b) Dining-Philosopher's problem

CODE

```
#include <stdio.h>
#include <stdlib.h>

int main() {
   int n, hungry, pos[10], choice;

   printf("Enter the total number of philosophers: ");
   scanf("%d", &n);
   printf("How many are hungry: ");
   scanf("%d", &hungry);

   for (int i = 0; i < hungry; i++) {
      printf("Enter philosopher %d position (1 to %d): ", i + 1, n);
      scanf("%d", &pos[i]);
   }

   while (1) {
      printf("\n1. One can eat at a time\n2. Two can eat at a time\n3. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &choice);

      if (choice == 1) {
         printf("Allow one philosopher to eat at any time\n");
         for (int i = 0; i < hungry; i++) {
            for (int j = 0; j < hungry; j++) {
               printf("P %d is waiting\n", pos[j]);
            }
            printf("P %d is granted to eat\n", pos[i]);
            printf("P %d has finished eating\n", pos[i]);
         }
      } else if (choice == 2) {
         printf("Allow two philosophers to eat at any time\n");
         for (int i = 0; i < hungry; i++) {
            printf("P %d is waiting\n", pos[i]);
         }
```

```
 for (int i = 0; i < hungry; i += 2) {
            if (i + 1 < hungry) {
               printf("P %d and P %d are granted to eat\n", pos[i], pos[i+1]);
               printf("P %d and P %d have finished eating\n", pos[i], pos[i+1]);
            } else {
               printf("P %d is granted to eat\n", pos[i]);
               printf("P %d has finished eating\n", pos[i]);
            }
         }
      } else if (choice == 3) {
         exit(0);
      }
   }

   return 0;
}
```

Output

```
Enter the total number of philosophers: 5
How many are hungry: 2
Enter philosopher 1 position (1 to 5): 2 3 1 4 5
Enter philosopher 2 position (1 to 5):
1. One can eat at a time
2. Two can eat at a time
3. Exit
Enter your choice: Allow one philosopher to eat at any time
P 2 is waiting
P 3 is waiting
P 2 is granted to eat
P 2 has finished eating
P 2 is waiting
P 3 is waiting
P 3 is granted to eat
P 3 has finished eating

1. One can eat at a time
2. Two can eat at a time
3. Exit
```

Program 5

Write a C program to simulate:
a) Bankers' algorithm for the purpose of deadlock avoidance.

CODE

```
#include <stdio.h>

int main()
{
int n, r, i, j, k;

printf("Enter number of processes\t--\t");
scanf("%d", &n);

printf("Enter number of resources\t--\t");
scanf("%d", &r);

int alloc[n][r], max[n][r], need[n][r];
int avail[r], work[r], finish[n], safe[n];


for(i = 0; i < n; i++)
{
printf("\nEnter details for P%d\n", i);

printf("Enter allocation\t--\t");
for(j = 0; j < r; j++)
{
scanf("%d", &alloc[i][j]);
}

printf("Enter Max\t\t--\t");
for(j = 0; j < r; j++)
{
scanf("%d", &max[i][j]);
}
}
```

```
printf("\nEnter Available Resources --\t");
for(i = 0; i < r; i++)
{
scanf("%d", &avail[i]);
work[i] = avail[i];
}


int pid;
int request[r];

printf("\nEnter New Request Details --\n");

printf("Enter pid\t--\t");
scanf("%d", &pid);

printf("Enter Request for Resources\t--\t");
for(i = 0; i < r; i++)
{
scanf("%d", &request[i]);
}


for(i = 0; i < r; i++)
{
avail[i] -= request[i];
alloc[pid][i] += request[i];
}


for(i = 0; i < n; i++)
{
for(j = 0; j < r; j++)
{
need[i][j] = max[i][j] - alloc[i][j];
}
}


for(i = 0; i < n; i++)
```

```
{
finish[i] = 0;
}

int count = 0;

printf("\nOUTPUT\n");

while(count < n)
{
int found = 0;

for(i = 0; i < n; i++)
{
if(finish[i] == 0)
{
int flag = 1;

for(j = 0; j < r; j++)
{
if(need[i][j] > work[j])
{
flag = 0;
break;
}
}

if(flag)
{
for(k = 0; k < r; k++)
{
work[k] += alloc[i][k];
}

printf("P%d is visited (", i);

for(k = 0; k < r; k++)
{
printf("%d", work[k]);

if(k != r - 1)
printf(" ");
```

```
}

printf(")\n");

safe[count++] = i;
finish[i] = 1;
found = 1;
}
}
}

if(found == 0)
{
break;
}
}


if(count == n)
{
printf("SYSTEM IS IN SAFE STATE\n");

printf("The Safe Sequence is -- (");

for(i = 0; i < n; i++)
{
printf("P%d", safe[i]);

if(i != n - 1)
printf(" ");
}

printf(")\n");
}
else
{
printf("SYSTEM IS NOT IN SAFE STATE\n");
}


printf("\n");
```

```
printf("Process\t\tAllocation\t\tMax\t\tNeed\n");

for(i = 0; i < n; i++)
{
printf("P%d\t\t", i);


for(j = 0; j < r; j++)
{
printf("%d ", alloc[i][j]);
}

printf("\t\t");


for(j = 0; j < r; j++)
{
printf("%d ", max[i][j]);
}

printf("\t\t");

for(j = 0; j < r; j++)
{
printf("%d ", need[i][j]);
}

printf("\n");
}

return 0;
}
```

Output

```
Enter number of processes       --      5
Enter number of resources       --      3

Enter details for P0
Enter allocation        --      0 1 0
Enter Max               --      7 5 3

Enter details for P1
Enter allocation        --      2 0 0
Enter Max               --      3 2 2

Enter details for P2
Enter allocation        --      3 0 2
Enter Max               --      9 0 2

Enter details for P3
Enter allocation        --      2 1 1
Enter Max               --      2 2 2

Enter details for P4
Enter allocation        --      0 0 2
Enter Max               --      4 3 3

Enter Available Resources --    3 3 2

Enter New Request Details --
Enter pid       --      1
Enter Request for Resources     --      1 0 2

OUTPUT
P1 is visited (6 3 4)
P2 is visited (9 3 6)
P3 is visited (11 4 7)
P4 is visited (11 4 9)
P0 is visited (11 5 9)
SYSTEM IS IN SAFE STATE
The Safe Sequence is -- (P1 P2 P3 P4 P0)

Process         Allocation              Max             Need
P0              0 1 0           7 5 3           7 4 3
P1              3 0 2           3 2 2           0 2 0
P2              3 0 2           9 0 2           6 0 0
P3              2 1 1           2 2 2           0 1 1
P4              0 0 2           4 3 3           4 3 1

Process returned 0 (0x0)   execution time : 77.894 s
Press any key to continue.
```

b) Deadlock Detection

CODE

```
#include <stdio.h>

#define MAX 10

int main()
{
int n, m;
int alloc[MAX][MAX], request[MAX][MAX];
int avail[MAX];

int finish[MAX];
int work[MAX];

int i, j;

printf("Enter number of processes -- ");
scanf("%d", &n);

printf("Enter number of resources -- ");
scanf("%d", &m);

printf("\nEnter Allocation Matrix:\n");

for(i = 0; i < n; i++)
{
printf("Allocation for P%d -- ", i);

for(j = 0; j < m; j++)
scanf("%d", &alloc[i][j]);
}


printf("\nEnter Request Matrix:\n");

for(i = 0; i < n; i++)
{
printf("Request for P%d -- ", i);
```

```
for(j = 0; j < m; j++)
scanf("%d", &request[i][j]);
}


printf("\nEnter Available Resources -- ");

for(i = 0; i < m; i++)
scanf("%d", &avail[i]);


printf("\n\nPROCESS\tALLOCATION\tREQUEST\n");

for(i = 0; i < n; i++)
{
printf("P%d\t", i);

for(j = 0; j < m; j++)
printf("%d ", alloc[i][j]);

printf("\t\t");


for(j = 0; j < m; j++)
printf("%d ", request[i][j]);

printf("\n");
}

printf("\nAVAILABLE RESOURCES: ");

for(i = 0; i < m; i++)
printf("%d ", avail[i]);

printf("\n");

for(i = 0; i < m; i++)
work[i] = avail[i];


for(i = 0; i < n; i++)
{
```

```
int empty = 1;

for(j = 0; j < m; j++)
{
if(alloc[i][j] != 0)
{
empty = 0;
break;
}
}

if(empty)
finish[i] = 1;
else
finish[i] = 0;
}


int changed = 1;

while(changed)
{
changed = 0;

for(i = 0; i < n; i++)
{
if(finish[i] == 0)
{
int possible = 1;

for(j = 0; j < m; j++)
{
if(request[i][j] > work[j])
{
possible = 0;
break;
}
}

if(possible)
{
printf("\nP%d can execute", i);
```

```
for(j = 0; j < m; j++)
work[j] += alloc[i][j];

printf("\nAvailable becomes: ");

for(j = 0; j < m; j++)
printf("%d ", work[j]);

printf("\n");

finish[i] = 1;
changed = 1;
}
}
}
}


int deadlock = 0;

for(i = 0; i < n; i++)
{
if(finish[i] == 0)
{
deadlock = 1;
break;
}
}

if(deadlock)
{
printf("\nDEADLOCK DETECTED\n");
printf("Processes involved in deadlock are: ");

for(i = 0; i < n; i++)
{
if(finish[i] == 0)
printf("P%d ", i);
}

printf("\n");
```

```
}
else
{
printf("\nNO DEADLOCK DETECTED\n");
}

return 0;
}
```

Output

```
Enter number of processes -- 5
Enter number of resources -- 3

Enter Allocation Matrix:
Allocation for P0 -- 0 1 0
Allocation for P1 -- 2 0 0
Allocation for P2 -- 3 0 3
Allocation for P3 -- 2 1 1
Allocation for P4 -- 0 0 2

Enter Request Matrix:
Request for P0 -- 0 0 0
Request for P1 -- 2 0 2
Request for P2 -- 0 0 0
Request for P3 -- 1 0 0
Request for P4 -- 0 0 2

Enter Available Resources -- 0 0 0


PROCESS ALLOCATION      REQUEST
P0      0 1 0           0 0 0
P1      2 0 0           2 0 2
P2      3 0 3           0 0 0
P3      2 1 1           1 0 0
P4      0 0 2           0 0 2

AVAILABLE RESOURCES: 0 0 0

P0 can execute
Available becomes: 0 1 0

P2 can execute
Available becomes: 3 1 3

P3 can execute
Available becomes: 5 2 4

P4 can execute
Available becomes: 5 2 6

P1 can execute
Available becomes: 7 2 6

NO DEADLOCK DETECTED

Process returned 0 (0x0)   execution time : 95.850 s
Press any key to continue.
```

Program 6

Write a C program to simulate the following contiguous memory allocation techniques.
a) Worst-fit
b) Best-fit
c) First-fit

CODE

```
#include <stdio.h>

void simulate(int b[], int n, int p[], int m, int type) {
    int alloc[20], occupied[20] = {0};
    for (int i = 0; i < m; i++) alloc[i] = -1;

    for (int i = 0; i < m; i++) {
        int idx = -1;
        for (int j = 0; j < n; j++) {
            if (!occupied[j] && b[j] >= p[i]) {
                if (type == 1) {
                    idx = j;
                    break;
                } else if (type == 2) {
                    if (idx == -1 || b[j] < b[idx]) idx = j;
                } else if (type == 3) {
                    if (idx == -1 || b[j] > b[idx]) idx = j;
                }
            }
        }
        if (idx != -1) {
            alloc[i] = idx;
            occupied[idx] = 1;
        }
    }

    printf("\nProcess No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < m; i++) {
        printf("%d\t\t%d\t\t", i + 1, p[i]);
        if (alloc[i] != -1) printf("%d\n", alloc[i] + 1);
        else printf("Not Allocated\n");
```

```
    }
}

int main() {
    int b[20], p[20], b_bak[20], n, m;

    printf("Enter number of blocks: ");
    scanf("%d", &n);
    printf("Enter size of each block:\n");
    for (int i = 0; i < n; i++) scanf("%d", &b[i]);

    printf("Enter number of processes: ");
    scanf("%d", &m);
    printf("Enter size of each process:\n");
    for (int i = 0; i < m; i++) scanf("%d", &p[i]);

    for (int i = 0; i < n; i++) b_bak[i] = b[i];
    printf("\n--- FIRST FIT ALLOCATION ---");
    simulate(b_bak, n, p, m, 1);

    for (int i = 0; i < n; i++) b_bak[i] = b[i];
    printf("\n--- BEST FIT ALLOCATION ---");
    simulate(b_bak, n, p, m, 2);

    for (int i = 0; i < n; i++) b_bak[i] = b[i];
    printf("\n--- WORST FIT ALLOCATION ---");
    simulate(b_bak, n, p, m, 3);

    return 0;
}
```

Output

```
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

FIRST FIT Allocation:
Process No      Process Size    Block No
1               212             2
2               417             5
3               112             2
4               426             Not Allocated

BEST FIT Allocation:
Process No      Process Size    Block No
1               212             4
2               417             2
3               112             3
4               426             5

WORST FIT Allocation:
Process No      Process Size    Block No
1               212             5
2               417             2
3               112             5
4               426             Not Allocated

Process returned 0 (0x0)   execution time : 31.851 s
Press any key to continue.
```

Program 7

Write a C program to simulate page replacement algorithms.
a) FIFO
b) LRU
c) Optimal

CODE

```
#include <stdio.h>

void run(int ref[], int n, int nf, int mode) {
   int frames[10], matrix[10][50], tracker[10] = {0}, fifo_idx = 0, faults = 0;
   for (int i = 0; i < nf; i++) frames[i] = -1;

   for (int i = 0; i < n; i++) {
      int found = -1;
      for (int j = 0; j < nf; j++) if (frames[j] == ref[i]) found = j;

      if (found != -1) {
         if (mode == 2) tracker[found] = i;
      }
 else {
         faults++;
         int slot = -1;
         for (int j = 0; j < nf; j++) if (frames[j] == -1) { slot = j; break; }

         if (slot == -1) {
            if (mode == 1) { slot = fifo_idx; fifo_idx = (fifo_idx + 1) % nf; }
            else {
               int opt_idx[10];
               for (int j = 0; j < nf; j++) {
                  opt_idx[j] = 1e9;
                  for (int k = i + 1; k < n; k++) if (frames[j] == ref[k]) { opt_idx[j]
= k; break; }
               }
               int target = 0;
               for (int j = 1; j < nf; j++) {
                  if (mode == 2 && tracker[j] < tracker[target]) target = j;
                  if (mode == 3 && opt_idx[j] > opt_idx[target]) target = j;
               }
```

```
                slot = target;
            }
        }

        frames[slot] = ref[i];
        if (mode == 2) tracker[slot] = i;
    }

    for (int j = 0; j < nf; j++) matrix[j][i] = frames[j];
  }

  for (int i = 0; i < nf; i++) {
    for (int j = 0; j < n; j++)
      if (matrix[i][j] != -1) printf("%d\t", matrix[i][j]); else printf(" \t");
    printf("\n");
  }
  printf("Total Page Faults: %d\n", faults);
}

int main() {
  int n, nf, ref[50];
  printf("Enter number of pages: "); scanf("%d", &n);
  printf("Enter reference string:\n");
  for (int i = 0; i < n; i++) scanf("%d", &ref[i]);
  printf("Enter number of frames: "); scanf("%d", &nf);

  printf("\n--- FIFO PAGE REPLACEMENT ---\n"); run(ref, n, nf, 1);
  printf("\n--- LRU PAGE REPLACEMENT ---\n");  run(ref, n, nf, 2);
  printf("\n--- OPTIMAL PAGE REPLACEMENT ---\n"); run(ref, n, nf, 3);
  return 0;
}
```

Output

```
Enter the number of Frames: 3
Enter the length of reference string: 6
Enter the reference string: 1 4 2 3 2 1

FIFO Page Replacement Process:
PF No. 1: 1 - -
PF No. 2: 1 4 -
PF No. 3: 1 4 2
PF No. 4: 3 4 2
PF No. 5: 3 1 2
FIFO Page Faults: 5

LRU Page Replacement Process:
PF No. 1: 1 - -
PF No. 2: 1 4 -
PF No. 3: 1 4 2
PF No. 4: 3 4 2
PF No. 5: 3 1 2
LRU Page Faults: 5

Optimal Page Replacement Process:
PF No. 1: 1 - -
PF No. 2: 1 4 -
PF No. 3: 1 4 2
PF No. 4: 1 3 2
Optimal Page Faults: 4

Process returned 0 (0x0)   execution time : 28.868 s
Press any key to continue.
```